

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:
Document → Chunking → Embedding → Retrieval → Generation
- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

9. Semantic Search over Multiple Documents: Create at least 10 documents and implement semantic search using embeddings. Retrieve the top three most relevant documents for a query.

Q1. Semantic Search over Multiple Documents

1. Aim

To develop a semantic search system using embeddings and FAISS that can search through multiple documents and retrieve the top three most relevant documents for a given user query.

2. Objective

The objective of this implementation is to understand how semantic search works using text embeddings and a vector database. Unlike traditional keyword-based search, semantic search identifies documents based on the meaning of the query. In this system, 10 documents related to Artificial Intelligence are converted into embeddings and stored in a FAISS vector database. When the user enters a query, the system finds and displays the three most relevant documents.

3. Technologies Used

- Python
- Google Colab
- Sentence Transformers
- all-MiniLM-L6-v2 embedding model
- FAISS
- NumPy

4. Documents Used

The system contains the following 10 documents:

1. Artificial Intelligence
2. Machine Learning
3. Deep Learning
4. Natural Language Processing
5. Computer Vision
6. Generative AI
7. Large Language Models
8. Retrieval-Augmented Generation
9. AI Agents
10. AI Ethics

5. Methodology

The semantic search system follows these steps:

Document Collection → Embedding Generation → FAISS Vector Database → User Query → Query Embedding → Semantic Search → Top 3 Relevant Documents

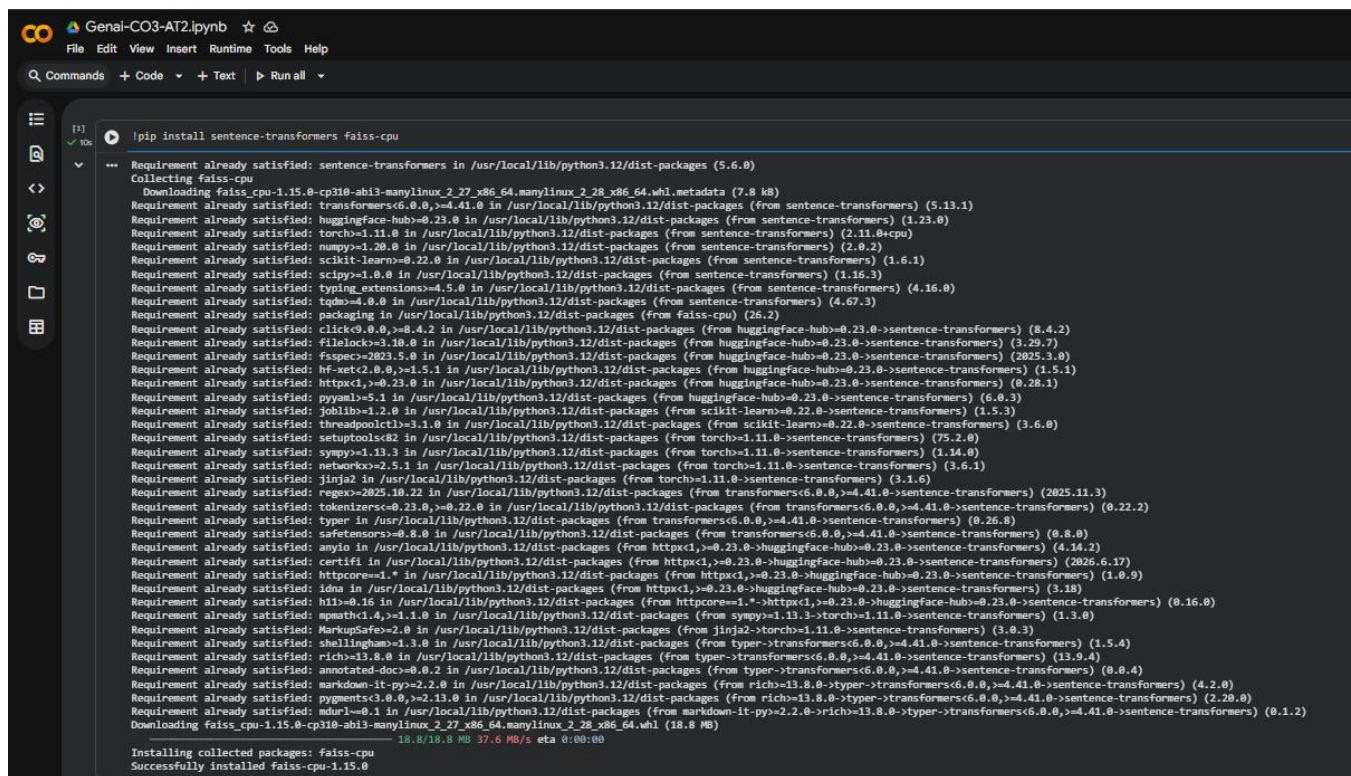
First, 10 documents related to Artificial Intelligence were created. The Sentence Transformer model all-MiniLM-L6-v2 was used to convert the documents into numerical embeddings. These embeddings represent the semantic meaning of the documents.

The generated embeddings were then stored in a FAISS vector database. When a user enters a query, the query is also converted into an embedding using the same model. FAISS compares the query embedding with the stored document embeddings and retrieves the three most relevant documents based on semantic similarity.

6. Implementation

The implementation was carried out using Python in Google Colab. Sentence Transformers was used for generating embeddings, while FAISS was used for efficient similarity search.

The system successfully generated embeddings for all 10 documents and stored them in the FAISS index. The user can enter different queries and the system retrieves the top three relevant documents along with their distance values and document content.

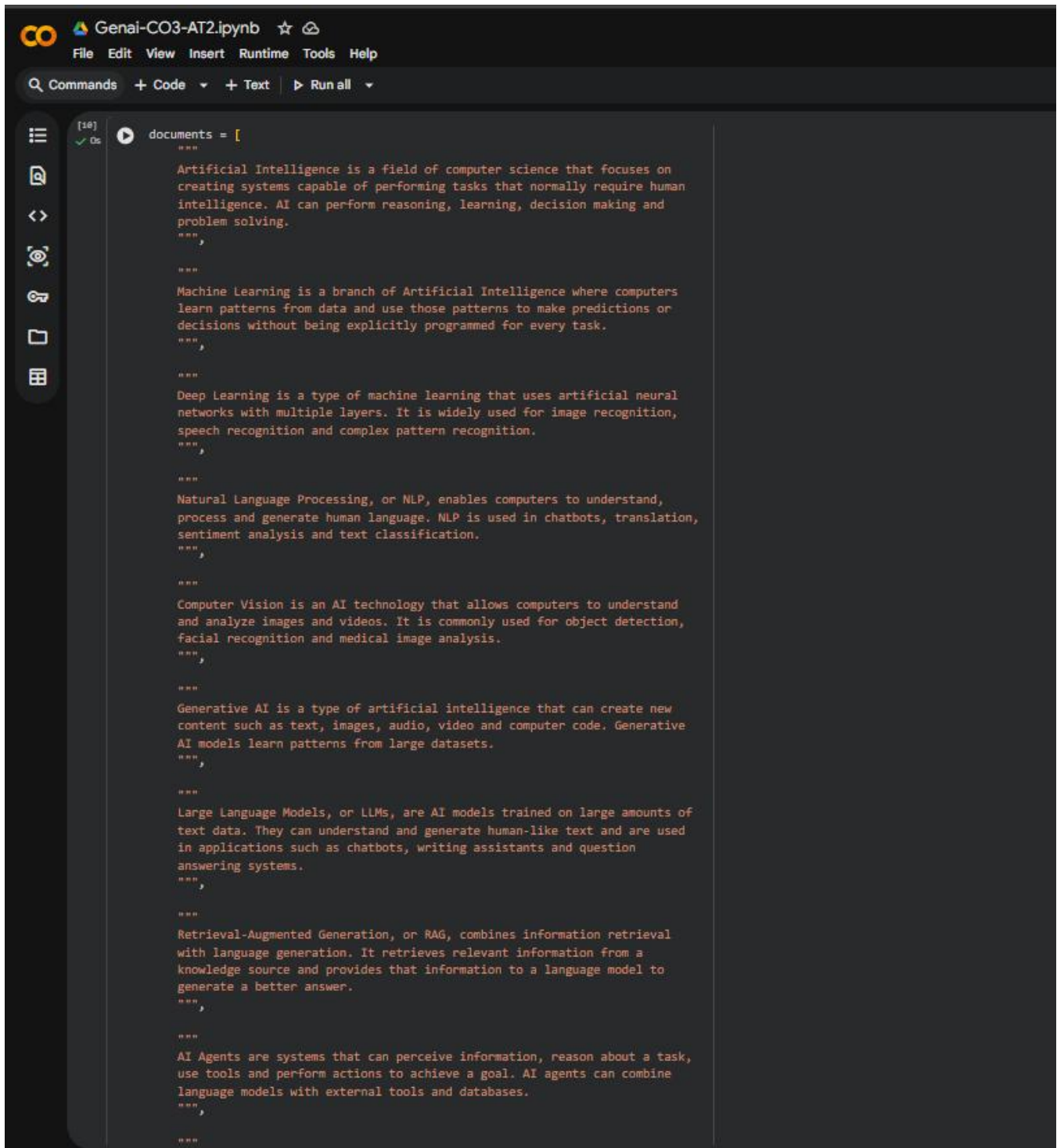


```
GenAI-CO3-AT2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

[1]
✓ 100%
!pip install sentence-transformers faiss-cpu

Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages (5.6.0)
Collecting faiss-cpu
  Downloading faiss_cpu-1.15.0-cp310-abi3-manylinux_2_27_x86_64_manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (5.13.1)
Requirement already satisfied: huggingface-hub<0.23.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.23.0)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.11.0+cpu)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.0.2)
Requirement already satisfied: scikit-learn>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.16.3)
Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.16.0)
Requirement already satisfied: tqdm>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.3)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (26.2)
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (8.4.2)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (3.29.7)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (2023.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (0.28.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<0.23.0->sentence-transformers) (6.0.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence-transformers) (3.6.0)
Requirement already satisfied: setuptools>=62 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.6.1)
Requirement already satisfied: ninja in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.1.6)
Requirement already satisfied: regex>=2023.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transformers) (2025.11.3)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transformers) (0.22.2)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transformers) (0.26.8)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence-transformers) (0.8.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<0.23.0->sentence-transformers) (4.14.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<0.23.0->sentence-transformers) (2026.6.17)
Requirement already satisfied: httpcore>=1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<0.23.0->sentence-transformers) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<0.23.0->sentence-transformers) (3.19)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<0.23.0->sentence-transformers) (0.16.0)
Requirement already satisfied: mmh3<4.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.11.0->sentence-transformers) (1.3.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (4.2.0)
Requirement already satisfied: pygments>=3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (2.20.0)
Requirement already satisfied: mdurl>=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers<6.0.0,>=4.41.0->sentence-transformers) (0.1.2)
Downloading faiss_cpu-1.15.0-cp310-abi3-manylinux_2_27_x86_64_manylinux_2_28_x86_64.whl (18.8 MB)
18.8/18.8 MB 37.6 MB/s eta 0:00:00
Installing collected packages: faiss-cpu
Successfully installed faiss-cpu-1.15.0
```

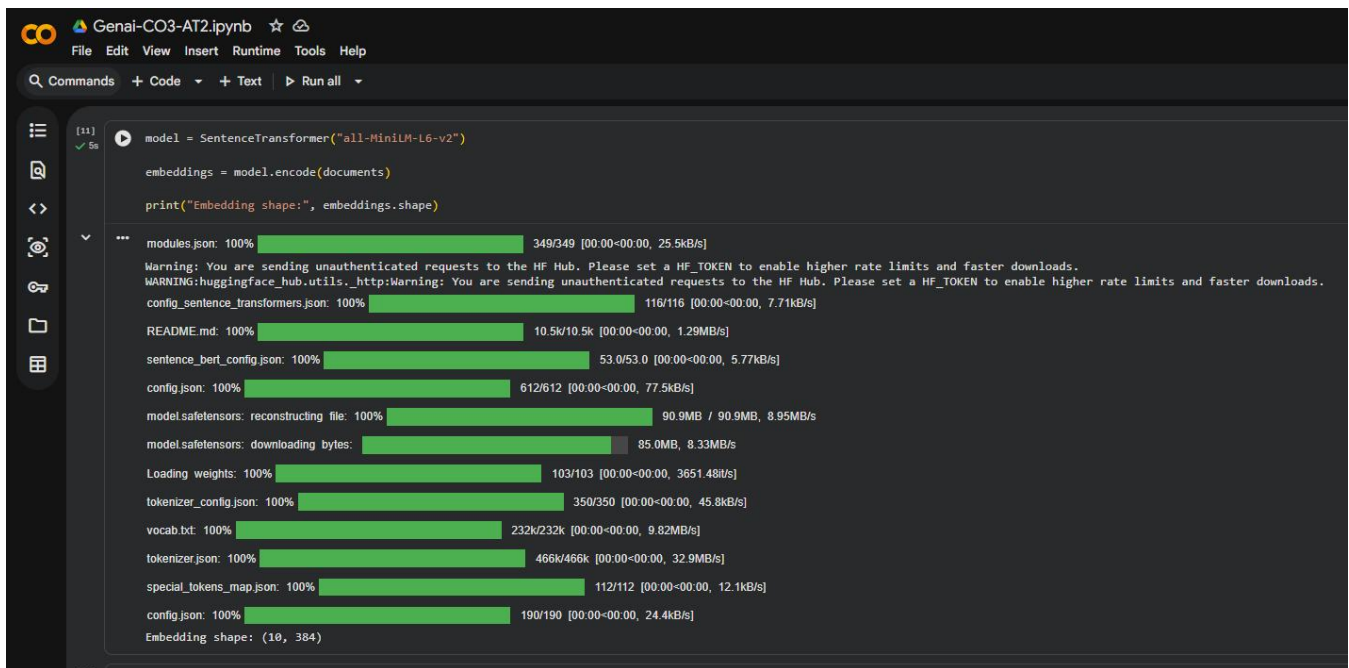
Fig 1: Import Libraries



The image shows a Jupyter Notebook interface with a dark theme. The top bar includes the logo, the filename 'Genai-CO3-AT2.ipynb', and standard menu items: File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all' buttons. On the left side, there is a vertical sidebar with icons for file management and execution. The main area displays a Python list named 'documents' containing 10 string elements, each representing a document about a different AI topic. The list is enclosed in triple quotes and separated by commas. The topics are: Artificial Intelligence, Machine Learning, Deep Learning, Natural Language Processing, Computer Vision, Generative AI, Large Language Models, Retrieval-Augmented Generation, and AI Agents. Each document string is indented within the list.

```
[10] documents = [  
    """  
    Artificial Intelligence is a field of computer science that focuses on  
    creating systems capable of performing tasks that normally require human  
    intelligence. AI can perform reasoning, learning, decision making and  
    problem solving.  
    """  
    ,  
    """  
    Machine Learning is a branch of Artificial Intelligence where computers  
    learn patterns from data and use those patterns to make predictions or  
    decisions without being explicitly programmed for every task.  
    """  
    ,  
    """  
    Deep Learning is a type of machine learning that uses artificial neural  
    networks with multiple layers. It is widely used for image recognition,  
    speech recognition and complex pattern recognition.  
    """  
    ,  
    """  
    Natural Language Processing, or NLP, enables computers to understand,  
    process and generate human language. NLP is used in chatbots, translation,  
    sentiment analysis and text classification.  
    """  
    ,  
    """  
    Computer Vision is an AI technology that allows computers to understand  
    and analyze images and videos. It is commonly used for object detection,  
    facial recognition and medical image analysis.  
    """  
    ,  
    """  
    Generative AI is a type of artificial intelligence that can create new  
    content such as text, images, audio, video and computer code. Generative  
    AI models learn patterns from large datasets.  
    """  
    ,  
    """  
    Large Language Models, or LLMs, are AI models trained on large amounts of  
    text data. They can understand and generate human-like text and are used  
    in applications such as chatbots, writing assistants and question  
    answering systems.  
    """  
    ,  
    """  
    Retrieval-Augmented Generation, or RAG, combines information retrieval  
    with language generation. It retrieves relevant information from a  
    knowledge source and provides that information to a language model to  
    generate a better answer.  
    """  
    ,  
    """  
    AI Agents are systems that can perceive information, reason about a task,  
    use tools and perform actions to achieve a goal. AI agents can combine  
    language models with external tools and databases.  
    """  
    ,  
    """  
    """  
]
```

Fig 2: Creating 10 documents



```
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(documents)
print("Embedding shape:", embeddings.shape)
```

modules.json: 100% 349/349 [00:00<00:00, 25.5kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 7.71kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 1.29MB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.77kB/s]

config.json: 100% 612/612 [00:00<00:00, 77.5kB/s]

model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.95MB/s

model.safetensors: downloading bytes: 85.0MB, 8.33MB/s

Loading weights: 100% 103/103 [00:00<00:00, 3651.48it/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 45.8kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 9.82MB/s]

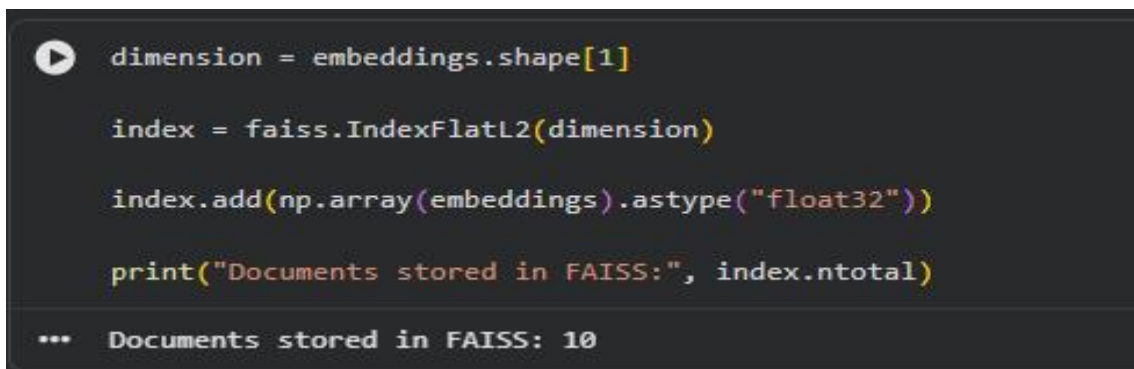
tokenizer.json: 100% 466k/466k [00:00<00:00, 32.9MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 12.1kB/s]

config.json: 100% 190/190 [00:00<00:00, 24.4kB/s]

Embedding shape: (10, 384)

Fig 3: Embedding Generation



```
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings).astype("float32"))
print("Documents stored in FAISS:", index.ntotal)
```

... Documents stored in FAISS: 10

Fig 4: FAISS database

7. Sample Query and Output

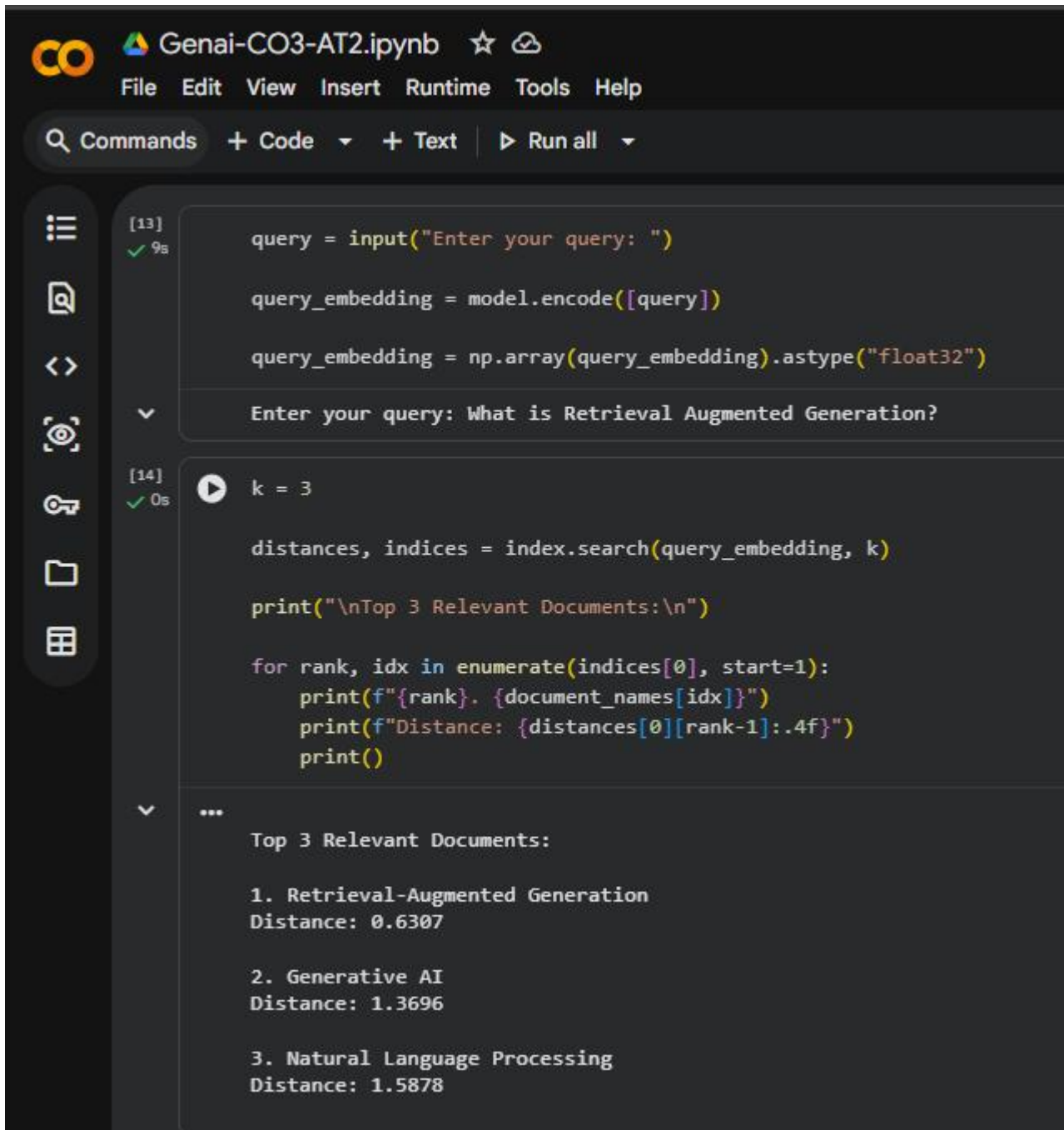
Query:

“What is Retrieval Augmented Generation?”

Retrieved Results:

- Retrieval-Augmented Generation
- Generative AI
- Natural Language Processing

The system successfully identified the documents that were semantically related to the given query and displayed their content.



The screenshot shows a Jupyter Notebook titled "Genai-CO3-AT2.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for commands, code, text, and running all cells. The notebook contains two code cells. The first cell, labeled [13], contains Python code for inputting a query, encoding it into an embedding, and converting it to float32. The input prompt is "Enter your query: ", and the user has entered "What is Retrieval Augmented Generation?". The second cell, labeled [14], contains code for searching the index with k=3, printing the top 3 relevant documents, and their distances. The output of the second cell shows the top 3 relevant documents and their distances:

```
[13]
✓ 9s

query = input("Enter your query: ")

query_embedding = model.encode([query])

query_embedding = np.array(query_embedding).astype("float32")

Enter your query: What is Retrieval Augmented Generation?

[14]
✓ 0s

k = 3

distances, indices = index.search(query_embedding, k)

print("\nTop 3 Relevant Documents:\n")

for rank, idx in enumerate(indices[0], start=1):
    print(f"{rank}. {document_names[idx]}")
    print(f"Distance: {distances[0][rank-1]:.4f}")
    print()

...

Top 3 Relevant Documents:

1. Retrieval-Augmented Generation
Distance: 0.6307

2. Generative AI
Distance: 1.3696

3. Natural Language Processing
Distance: 1.5878
```

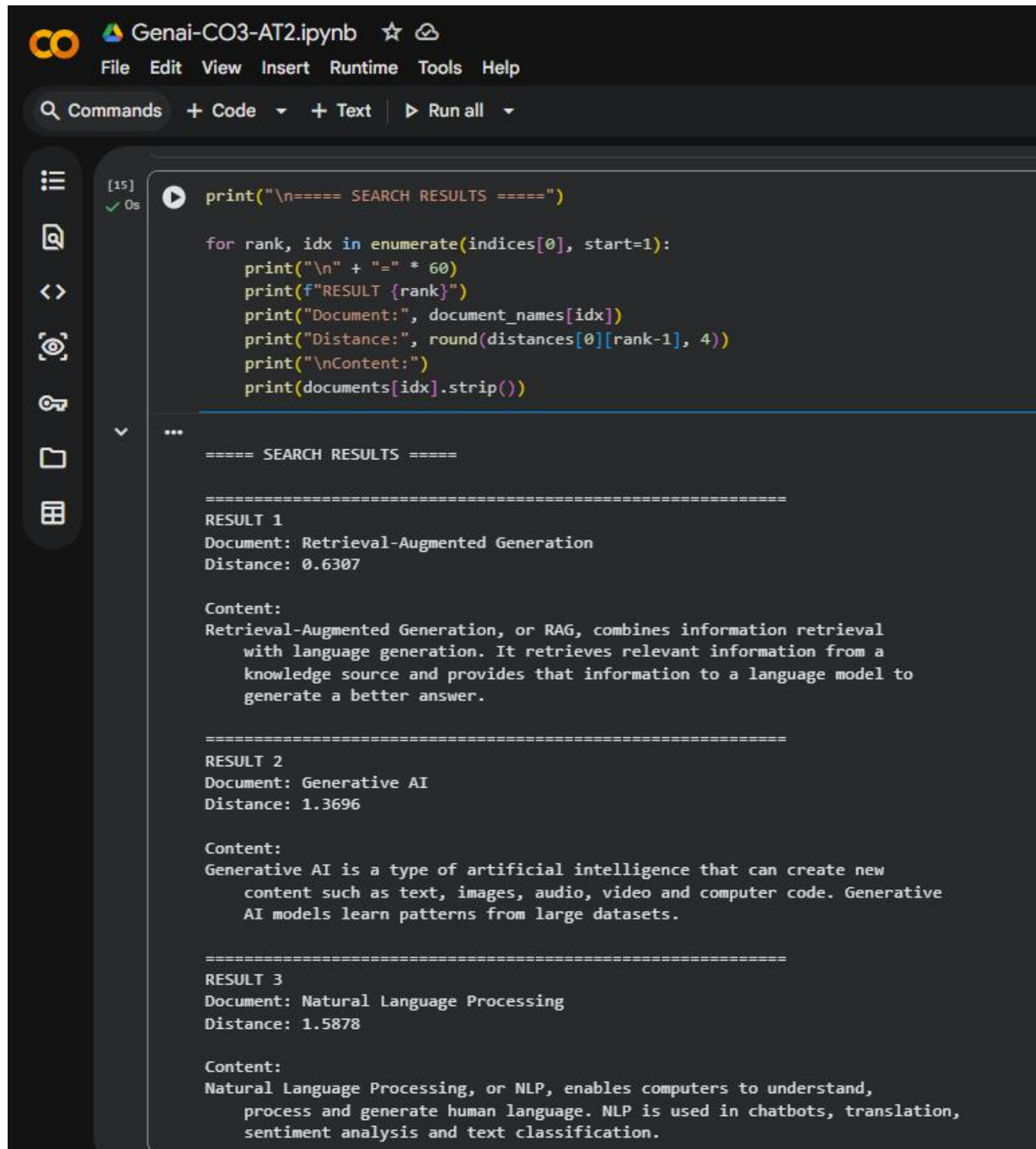
Figure 5: Semantic Search Query and Top-3 Relevant Documents

8. Multiple Query Testing

The system was tested with multiple queries to verify its semantic search capability. Example queries include:

- What is Retrieval Augmented Generation?
- What is machine learning?
- How do computers understand human language?
- What are the benefits of Generative AI?

For each query, the system retrieved the top three semantically relevant documents.



The screenshot shows a Jupyter Notebook titled "Genai-CO3-AT2.ipynb". The code cell contains a loop that iterates over the top three search results. The output displays the rank, document name, distance, and content for each result.

```
[15] ✓ 0s
print("\n===== SEARCH RESULTS =====")

for rank, idx in enumerate(indices[0], start=1):
    print("\n" + "=" * 60)
    print(f"RESULT {rank}")
    print("Document:", document_names[idx])
    print("Distance:", round(distances[0][rank-1], 4))
    print("Content:")
    print(documents[idx].strip())

...

===== SEARCH RESULTS =====

=====
RESULT 1
Document: Retrieval-Augmented Generation
Distance: 0.6307

Content:
Retrieval-Augmented Generation, or RAG, combines information retrieval
with language generation. It retrieves relevant information from a
knowledge source and provides that information to a language model to
generate a better answer.

=====
RESULT 2
Document: Generative AI
Distance: 1.3696

Content:
Generative AI is a type of artificial intelligence that can create new
content such as text, images, audio, video and computer code. Generative
AI models learn patterns from large datasets.

=====
RESULT 3
Document: Natural Language Processing
Distance: 1.5878

Content:
Natural Language Processing, or NLP, enables computers to understand,
process and generate human language. NLP is used in chatbots, translation,
sentiment analysis and text classification.
```

Figure 7: Top-3 Retrieved Documents and Their Content

9. Result

The semantic search system was successfully implemented using Sentence Transformer embeddings and FAISS. It was able to process user queries and retrieve the top three relevant documents based on semantic similarity.

10. Conclusion

This implementation demonstrates how embeddings and vector databases can be used to perform semantic search over multiple documents. The system can identify relevant information based on the meaning of a query rather than depending only on exact keyword matching. Thus, the required semantic search functionality was successfully achieved.

40. Complete RAG + AI Agent Demonstration: Develop a complete RAG-based AI Agent integrating document loading, chunking, embeddings, semantic search, FAISS/ChromaDB, retrieval, LLM generation, query handling, tool calling, source citation, and unsupported-query handling.

Q2. Complete RAG + AI Agent Demonstration

1. Aim

To develop a complete Retrieval-Augmented Generation (RAG) based AI Agent that can load documents, divide them into chunks, generate embeddings, store them in a FAISS vector database, retrieve relevant information, generate answers using an LLM, provide source citations, and handle unsupported queries.

2. Objective

The objective of this implementation is to develop an intelligent document question-answering system using Retrieval-Augmented Generation. The system retrieves relevant information from a collection of documents before generating an answer. This helps the system provide answers based on the available documents instead of depending only on the language model's general knowledge.

The system also demonstrates semantic search, vector database usage, LLM-based answer generation, source citation, multiple query handling, and unsupported-query handling.

3. Technologies Used

- Python
- Google Colab
- Google Gemini 3.6 Flash
- Sentence Transformers
- all-MiniLM-L6-v2
- FAISS
- NumPy
- Google GenAI SDK

4. Documents Used

The RAG system uses 10 documents related to Artificial Intelligence:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Natural Language Processing
- Computer Vision
- Generative AI
- Large Language Models
- Retrieval-Augmented Generation
- AI Agents
- AI Ethics

These documents form the knowledge base of the RAG system.

5. System Architecture

The proposed system follows the architecture:

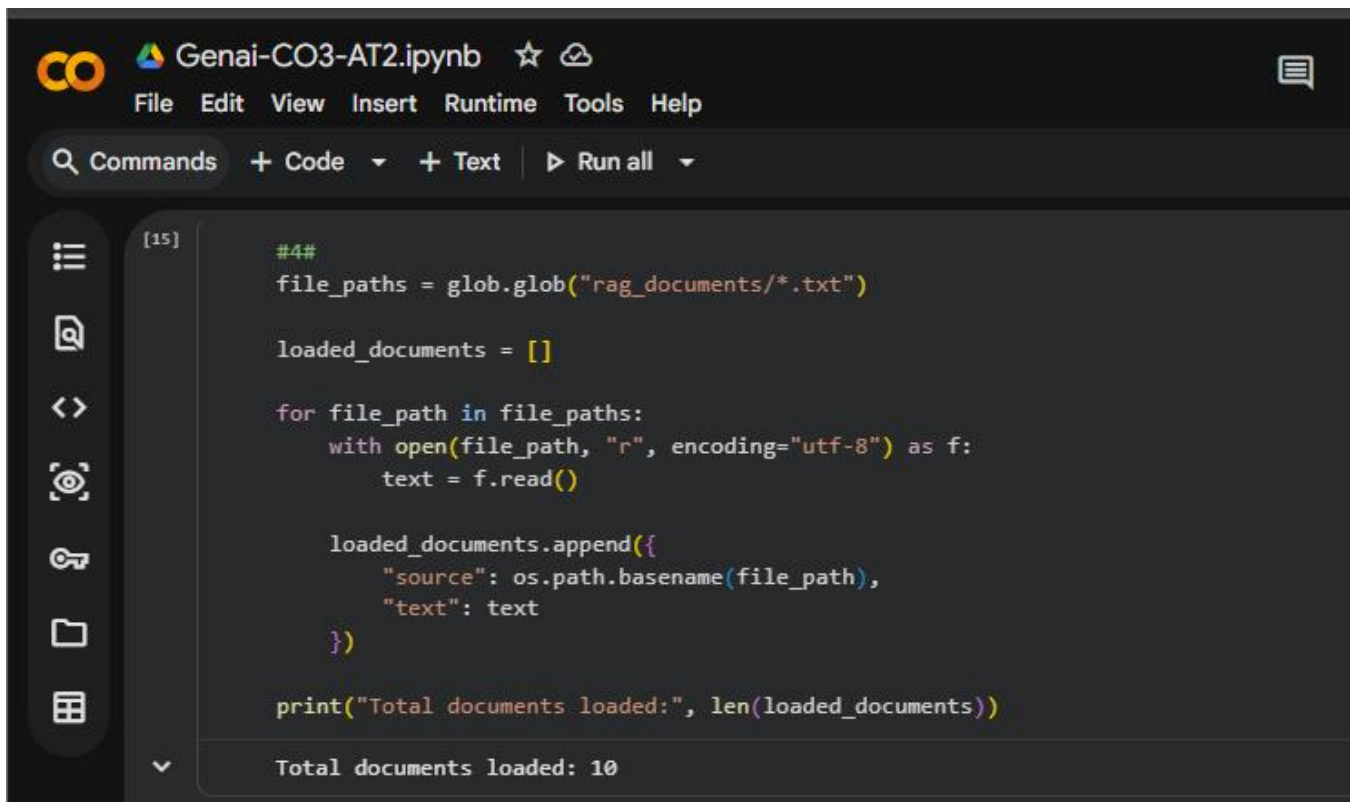
Documents → Document Loading → Chunking → Embeddings → FAISS Vector Database → User Query → Semantic Retrieval → Relevant Context → Gemini LLM → Generated Answer → Source Citation

The system first loads the documents and divides their content into smaller chunks. Each chunk is converted into a numerical embedding using the Sentence Transformer model. The embeddings are stored in a FAISS vector database.

When the user enters a question, the question is also converted into an embedding. FAISS performs a similarity search and retrieves the most relevant document chunks. These retrieved chunks are then provided as context to the Gemini language model. Gemini generates an answer based on the retrieved information. The system also displays the source documents used for generating the answer.

6. Document Loading

The documents were stored as individual text files in a `rag_documents` directory. Python was used to load the files and store their source names and contents.

The image shows a Jupyter Notebook interface with a dark theme. At the top, the title bar reads "Genai-CO3-AT2.ipynb" with a star icon and a refresh icon. Below the title bar is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar below the menu bar contains "Commands", "+ Code", "+ Text", and "Run all". On the left side, there is a vertical sidebar with icons for file explorer, search, and other functions. The main area displays a code cell labeled "[15]". The code in the cell is as follows:

```
#4#
file_paths = glob.glob("rag_documents/*.txt")

loaded_documents = []

for file_path in file_paths:
    with open(file_path, "r", encoding="utf-8") as f:
        text = f.read()

    loaded_documents.append({
        "source": os.path.basename(file_path),
        "text": text
    })

print("Total documents loaded:", len(loaded_documents))
```

Below the code, the output of the cell is displayed: "Total documents loaded: 10".

Figure 1: Loading Multiple Documents for RAG

The system successfully loaded all 10 documents into the RAG pipeline.

7. Document Chunking

After loading the documents, the text was divided into smaller chunks. Chunking helps the retrieval system search smaller and more meaningful sections of the documents.

An overlap was also used between consecutive chunks so that important information near the boundary of two chunks is not lost.

The chunks were stored along with their source document name and chunk number.



Q Commands + Code + Text ▶ Run all ▼

[16]
✓ 0s

```
#5#
def chunk_text(text, chunk_size=80, overlap=20):
    words = text.split()
    chunks = []

    start = 0

    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])

        if chunk.strip():
            chunks.append(chunk)

        start += chunk_size - overlap

    return chunks

chunks = []

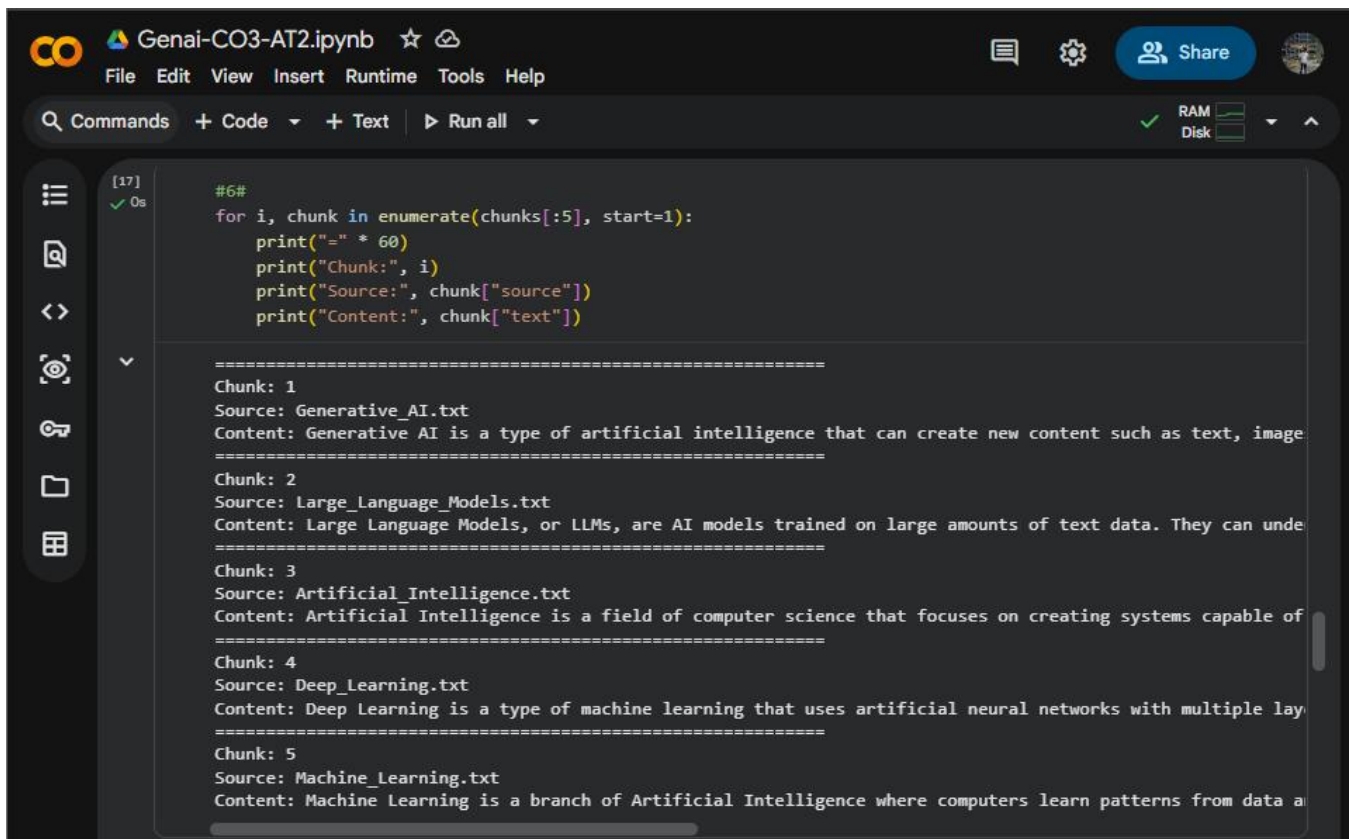
for doc in loaded_documents:
    doc_chunks = chunk_text(doc["text"])

    for i, chunk in enumerate(doc_chunks):
        chunks.append({
            "source": doc["source"],
            "chunk_id": i + 1,
            "text": chunk
        })

    print("Total chunks created:", len(chunks))
```



... Total chunks created: 10



The screenshot shows a Jupyter Notebook titled 'Genai-CO3-AT2.ipynb'. The code in the cell is as follows:

```
#6#
for i, chunk in enumerate(chunks[:5], start=1):
    print("=" * 60)
    print("Chunk:", i)
    print("Source:", chunk["source"])
    print("Content:", chunk["text"])
```

The output of the code is displayed below the code cell, showing five chunks of text with their sources and content. Each chunk is separated by a line of 60 equals signs.

```
=====  
Chunk: 1  
Source: Generative_AI.txt  
Content: Generative AI is a type of artificial intelligence that can create new content such as text, image  
=====  
Chunk: 2  
Source: Large_Language_Models.txt  
Content: Large Language Models, or LLMs, are AI models trained on large amounts of text data. They can unde  
=====  
Chunk: 3  
Source: Artificial_Intelligence.txt  
Content: Artificial Intelligence is a field of computer science that focuses on creating systems capable of  
=====  
Chunk: 4  
Source: Deep_Learning.txt  
Content: Deep Learning is a type of machine learning that uses artificial neural networks with multiple lay  
=====  
Chunk: 5  
Source: Machine_Learning.txt  
Content: Machine Learning is a branch of Artificial Intelligence where computers learn patterns from data a
```

8. Embedding Generation

The Sentence Transformer model all-MiniLM-L6-v2 was used to generate embeddings for the document chunks.

An embedding represents the semantic meaning of text as a numerical vector. These vectors allow the system to compare the meaning of a user's query with the meaning of the stored document chunks.

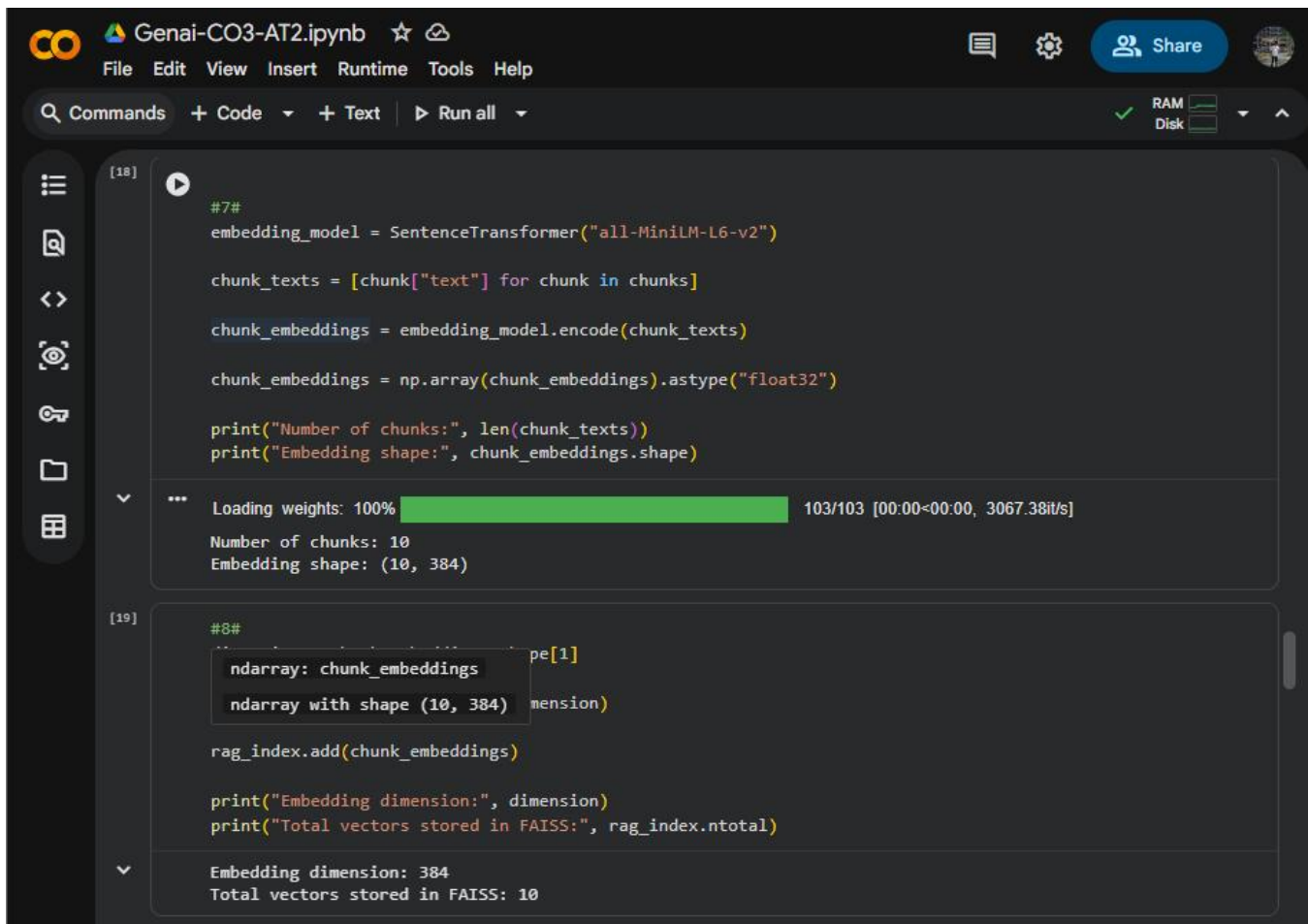
The generated embeddings were converted into numerical arrays and prepared for storage in the FAISS vector database.

9. FAISS Vector Database

FAISS was used as the vector database for storing the generated document embeddings.

The FAISS index performs similarity search between the query embedding and the stored document embeddings. The system retrieves the most relevant document chunks based on their similarity distance.

This allows the RAG system to perform efficient semantic retrieval.



The screenshot shows a Jupyter Notebook titled 'Genai-CO3-AT2.ipynb'. The interface includes a top bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help' menus. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all' buttons. On the right, there are icons for chat, settings, and a 'Share' button, along with RAM and Disk usage indicators.

The notebook contains two code cells. Cell [18] defines a SentenceTransformer model, processes document chunks into embeddings, and prints the number of chunks and embedding shape. Cell [19] adds these embeddings to a FAISS index and prints the embedding dimension and total vectors stored.

```
[18]
#7#
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

chunk_texts = [chunk["text"] for chunk in chunks]

chunk_embeddings = embedding_model.encode(chunk_texts)

chunk_embeddings = np.array(chunk_embeddings).astype("float32")

print("Number of chunks:", len(chunk_texts))
print("Embedding shape:", chunk_embeddings.shape)

... Loading weights: 100% 103/103 [00:00<00:00, 3067.38it/s]
Number of chunks: 10
Embedding shape: (10, 384)

[19]
#8#
rag_index.add(chunk_embeddings)

print("Embedding dimension:", dimension)
print("Total vectors stored in FAISS:", rag_index.ntotal)

Embedding dimension: 384
Total vectors stored in FAISS: 10
```

Embedding Generation for Document Chunks

10. Semantic Retrieval

When a user enters a question, the question is converted into an embedding using the same Sentence Transformer model.

The query embedding is then compared with the embeddings stored in FAISS. The system retrieves the top relevant document chunks.

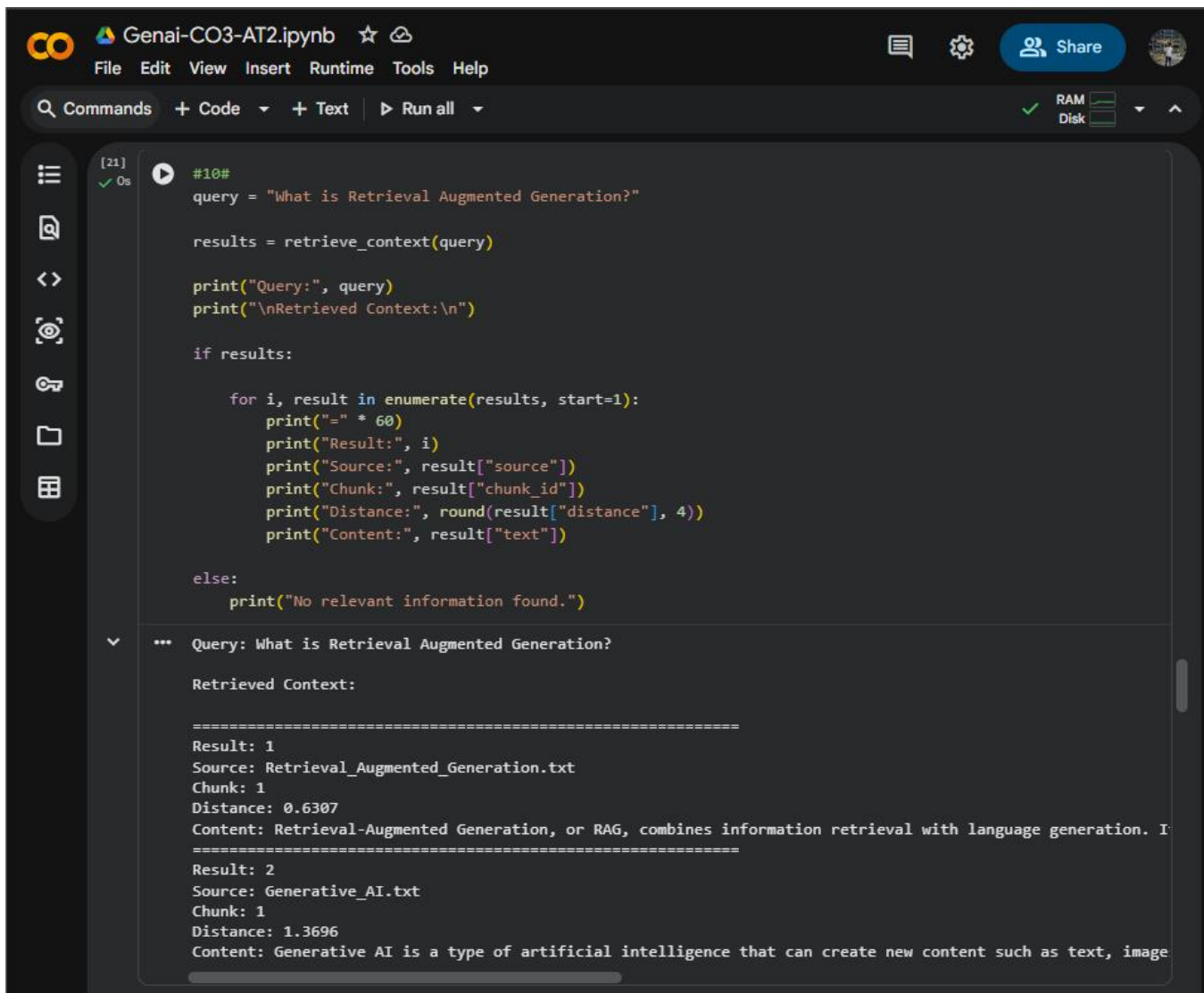
For example, for the query:

"What is Retrieval Augmented Generation?"

the system retrieved relevant information from documents such as:

- Retrieval-Augmented Generation
- Generative AI
- Large Language Models

The retrieved information is then used as context for generating the final answer.



```
[21] ✓ 0s #10#
query = "What is Retrieval Augmented Generation?"

results = retrieve_context(query)

print("Query:", query)
print("\nRetrieved Context:\n")

if results:
    for i, result in enumerate(results, start=1):
        print("=" * 60)
        print("Result:", i)
        print("Source:", result["source"])
        print("Chunk:", result["chunk_id"])
        print("Distance:", round(result["distance"], 4))
        print("Content:", result["text"])
    else:
        print("No relevant information found.")

*** Query: What is Retrieval Augmented Generation?

Retrieved Context:

=====
Result: 1
Source: Retrieval_Augmented_Generation.txt
Chunk: 1
Distance: 0.6307
Content: Retrieval-Augmented Generation, or RAG, combines information retrieval with language generation. I
=====
Result: 2
Source: Generative_AI.txt
Chunk: 1
Distance: 1.3696
Content: Generative AI is a type of artificial intelligence that can create new content such as text, image
```

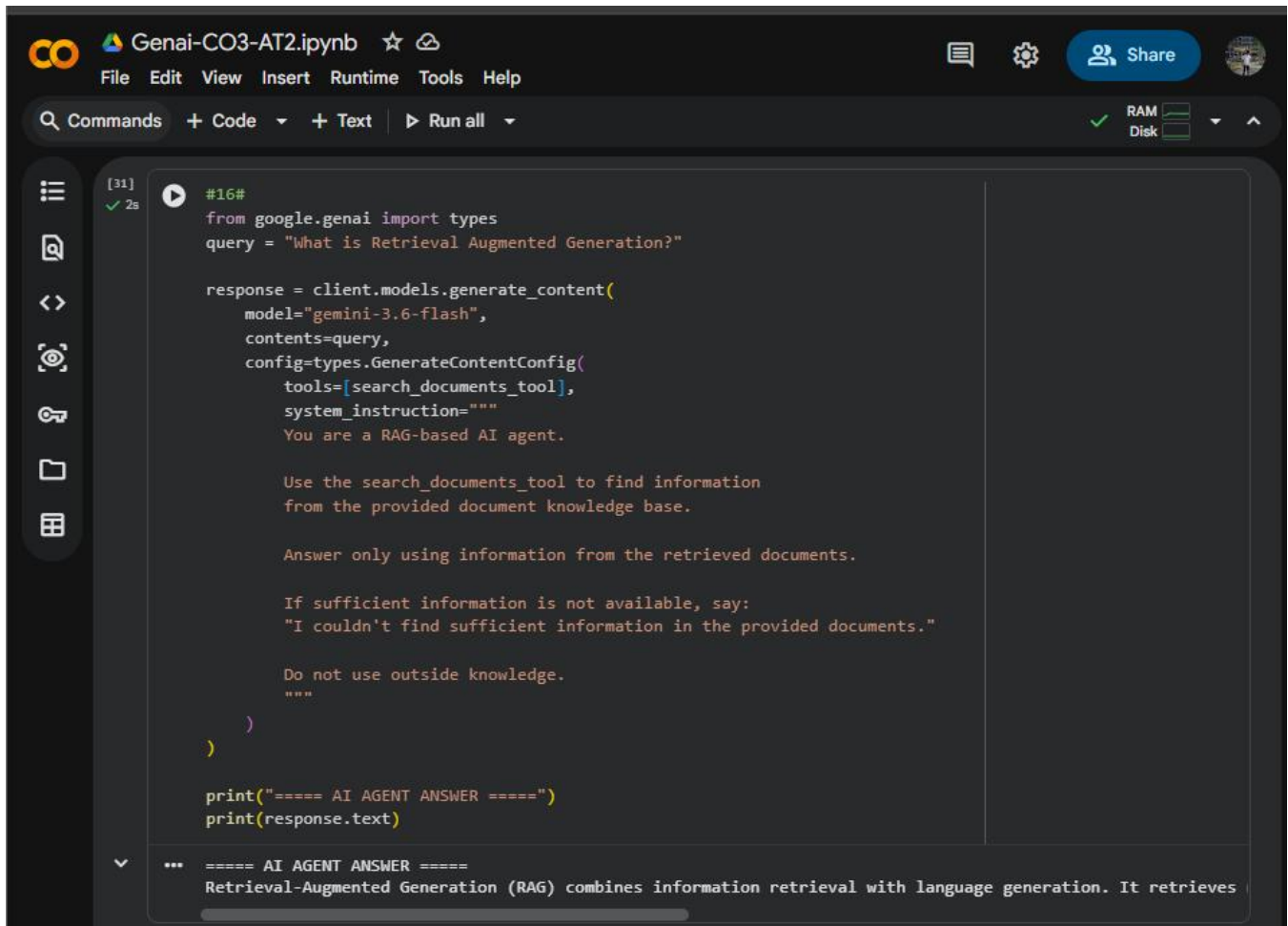
Semantic Retrieval of Relevant Document Chunks

11. Gemini LLM Integration

Google Gemini 3.6 Flash was integrated into the system as the Large Language Model.

The retrieved document information is provided to Gemini as context. The model generates an answer based on the retrieved information.

The system instruction also directs the model to answer using the provided document information and avoid using unsupported external information.



```
#16#
from google.genai import types
query = "What is Retrieval Augmented Generation?"

response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents=query,
    config=types.GenerateContentConfig(
        tools=[search_documents_tool],
        system_instruction="""
        You are a RAG-based AI agent.

        Use the search_documents_tool to find information
        from the provided document knowledge base.

        Answer only using information from the retrieved documents.

        If sufficient information is not available, say:
        "I couldn't find sufficient information in the provided documents."

        Do not use outside knowledge.
        """
    )
)

print("==== AI AGENT ANSWER =====")
print(response.text)
```

... ===== AI AGENT ANSWER =====
Retrieval-Augmented Generation (RAG) combines information retrieval with language generation. It retrieves

Gemini-Based RAG Answer Generation

12. RAG AI Agent and Tool Calling

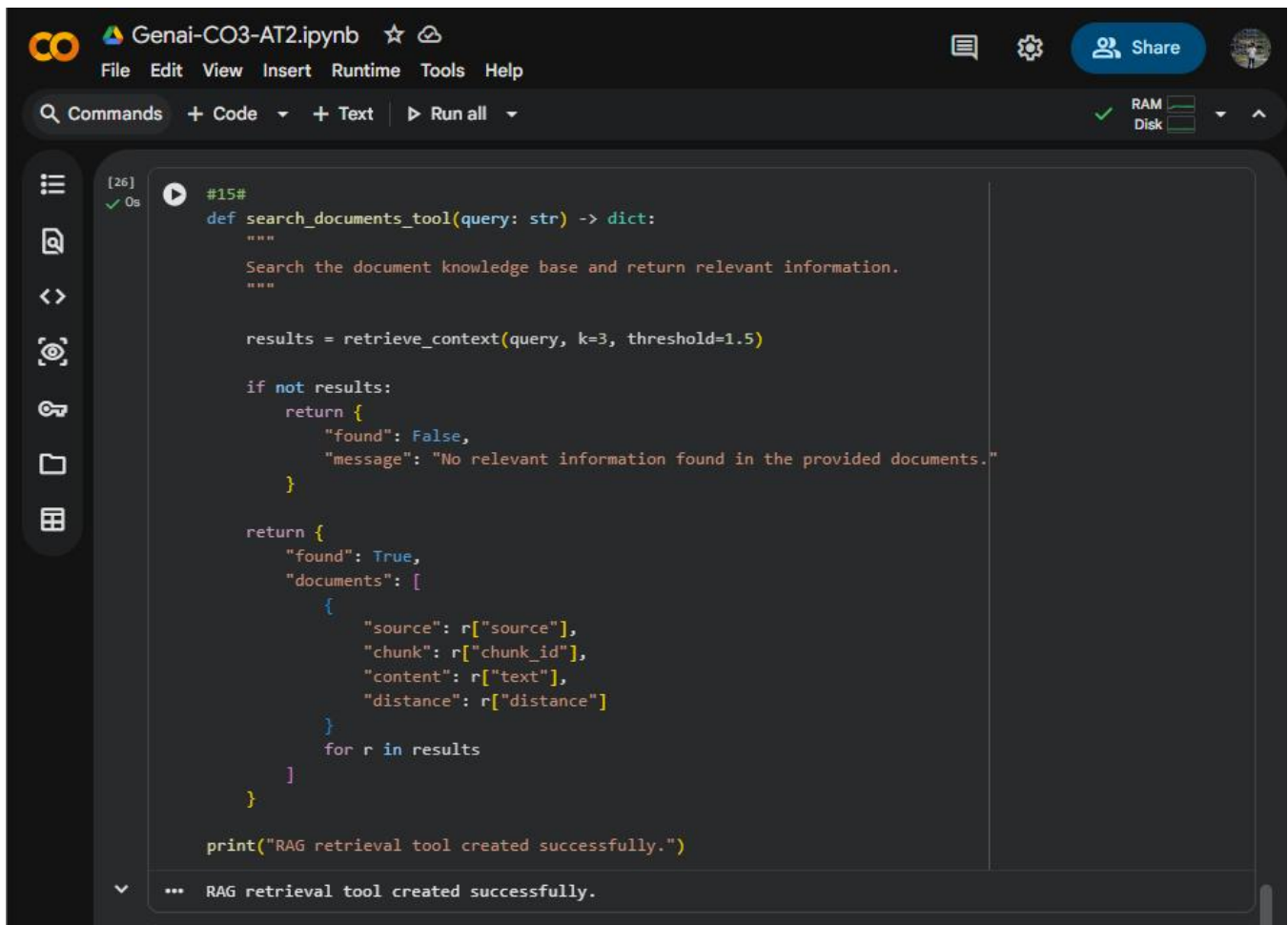
A document retrieval function was implemented as a tool for the AI Agent.

The tool searches the FAISS knowledge base whenever relevant information is required. The AI Agent can use this retrieval tool to obtain information from the documents before generating an answer.

The overall process is:

User Query → AI Agent → Retrieval Tool → FAISS → Relevant Documents → Gemini → Final Answer

This demonstrates the integration of retrieval and language generation in an AI Agent.



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the 'Genai-CO3-AT2.ipynb' title, a star icon, and a 'Share' button. Below the title bar is a menu with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar shows 'Commands', '+ Code', '+ Text', and 'Run all'. On the right, there are RAM and Disk usage indicators. The left sidebar contains icons for file management and search. The main area displays a Python code cell with the following content:

```
[26] #15#
✓ 0s
def search_documents_tool(query: str) -> dict:
    """
    Search the document knowledge base and return relevant information.
    """

    results = retrieve_context(query, k=3, threshold=1.5)

    if not results:
        return {
            "found": False,
            "message": "No relevant information found in the provided documents."
        }

    return {
        "found": True,
        "documents": [
            {
                "source": r["source"],
                "chunk": r["chunk_id"],
                "content": r["text"],
                "distance": r["distance"]
            }
            for r in results
        ]
    }

print("RAG retrieval tool created successfully.")
```

Below the code cell, a status bar shows '... RAG retrieval tool created successfully.'

RAG Retrieval Tool Used by the AI Agent

13. Source Citation

The system displays the source documents used during retrieval along with the generated answer.

For example, when answering a question about Retrieval-Augmented Generation, the system displays sources such as:

- Retrieval_Augmented_Generation.txt
- Generative_AI.txt

The source information helps the user understand where the retrieved information came from and improves the transparency and groundedness of the generated answer.

14. Unsupported Query Handling

The system was also tested with queries that are not related to the information available in the document collection.

For unsupported questions, the system checks whether sufficiently relevant information is available in the knowledge base. If suitable information is not found, the system responds:

"I couldn't find sufficient information in the provided documents."

This prevents the system from providing unsupported answers when the required information is not present in the document collection.

15. Multiple Query Testing

The RAG AI Agent was tested using multiple queries to verify its performance.

The following queries were tested:

- **What is Generative AI?**
- **What are Large Language Models?**
- **How does Retrieval Augmented Generation work?**
- **What is Computer Vision?**

For each query, the system retrieved relevant document information, generated an answer using Gemini, and displayed the source documents used.

The successful results demonstrate that the system can handle different queries over the same document knowledge base.

```
=====
... QUERY: What is Generative AI?

ANSWER:
Based on the provided documents, Generative AI is a type of artificial intelligence that can create new con

SOURCES:
- Generative_AI.txt
- Artificial_Intelligence.txt
- AI_Agents.txt

=====
QUERY: What are Large Language Models?

ANSWER:
Based on the provided documents, Large Language Models (LLMs) are AI models trained on large amounts of tex

SOURCES:
- Large_Language_Models.txt
- AI_Agents.txt
- Natural_Language_Processing.txt

=====
QUERY: How does Retrieval Augmented Generation work?

ANSWER:
Based on the provided documents, Retrieval-Augmented Generation (RAG) works by combining information retrie

SOURCES:
- Retrieval_Augmented_Generation.txt
- Generative_AI.txt

=====
QUERY: What is Computer Vision?

ANSWER:
Computer Vision is an AI technology that allows computers to understand and analyze images and videos. It i

SOURCES:
- Computer_Vision.txt
- Machine_Learning.txt
- Artificial_Intelligence.txt
```

Multiple Query Testing of the RAG AI Agent

16. End-to-End Workflow

The complete workflow of the developed system is:

Step 1: Load the 10 documents.

Step 2: Divide the documents into smaller chunks.

Step 3: Generate embeddings using Sentence Transformer.

Step 4: Store the embeddings in FAISS.

Step 5: Accept a user query.

Step 6: Convert the query into an embedding.

Step 7: Perform semantic similarity search using FAISS.

Step 8: Retrieve the relevant document chunks.

Step 9: Provide the retrieved information to Gemini.

Step 10: Generate a grounded answer.

Step 11: Display the source documents.

Step 12: Handle unsupported queries appropriately.

17. Result

The complete RAG-based AI Agent was successfully implemented using Python, Sentence Transformers, FAISS, and Gemini 3.6 Flash.

The system successfully loads documents, performs chunking, generates embeddings, stores them in FAISS, retrieves relevant information, generates answers using the LLM, displays source documents, and handles multiple queries.

The implementation demonstrates a complete RAG pipeline from document processing to answer generation.

18. Conclusion

The RAG-based AI Agent successfully demonstrates how document retrieval and Large Language Models can be combined to build an intelligent question-answering system.

The use of embeddings and FAISS enables semantic retrieval of relevant information, while Gemini generates answers using the retrieved context. Source citation improves transparency, and unsupported-query handling reduces the possibility of generating answers when sufficient information is not available.

Therefore, the developed system demonstrates the complete Retrieval-Augmented Generation workflow along with introductory AI Agent functionality.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed

Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						